ЧЕМПИОНАТ «ВПОТОКЕ» · КЕЙС №2 · ПРИЛОЖЕНИЕ К ПРЕЗЕНТАЦИИ

Кредитный конвейер: от заявки до решения

Команда SAD! · Рахматуллин Динар · Чиранов Александр · Коломеец Софья · Игнатьев Дмитрий · Иванов Юрий

91/100	12	260	0
совпадений с разметкой applications.csv, 9 расхождений объяснены	правил решения строго по порядку, балл 0–100	автотестов + 18/18 живых сценариев	номеров паспорта на сервере: только отпечаток (OPRF)

Банк «Фин-Тех» запускает онлайн-кредитование. Клиент сам подаёт заявку в приложении банка (вход по номеру телефона с подтверждением звонком), конвейер проверяет данные в СМЭВ и БКИ (эмуляторы по спецификации кейса), считает балл и применяет правила, а затем выдаёт **предварительное решение**: одобрено (с суммой, сроком, ставкой и справкой с цифровой подписью), отказ (с понятной причиной и встречным предложением, если отказ можно исправить параметрами), «нужны данные» или ручная проверка андеррайтером.

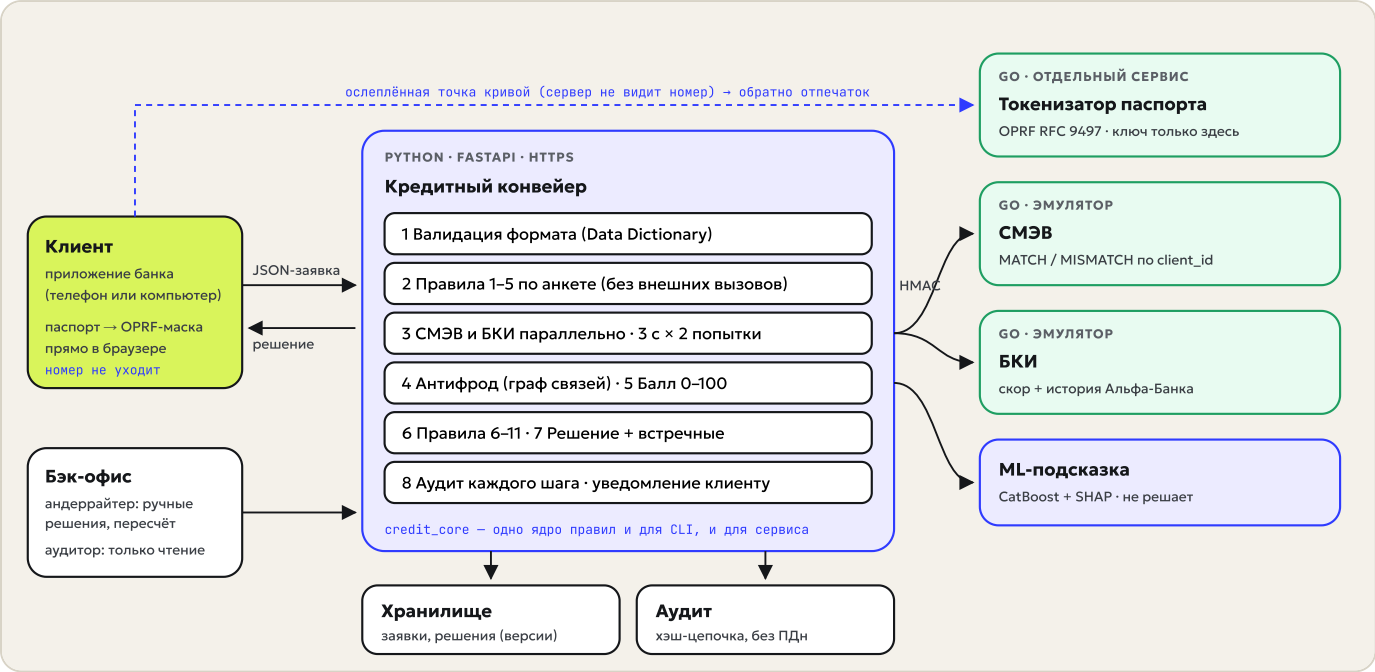
> **Как проверить самим.** Сайт: `https://credit.fashf.ru`. Код: `github.com/fashfffffff/sad-credit-pipeline`. Консольный прототип: `python prototype/04_прототип.py data/case2/applications.csv --out decisions.csv` (только стандартная библиотека Python).

Содержание: A.1 схема системы · A.2 ER-модель · A.3 схема решения · B.1 правила и пороги · B.2 ситуация → reason → решение · B.3 тесты и расхождения · B.4 интеграция, безопасность, экраны · C журнал промптов · D матрица вклада · E артефакты

Что нашли в материалах организаторов

product.csv	версия от 07.11: 3 продукта, подчёркивания экранированы как в Markdown (`product\_id`, `prod\_standard`) — коды не совпадают с остальными материалами. Убрали обратные слеши, оригинал сохранили. (Версия до 07.11 была Excel-книгой с расширением .csv.)
Разметка CSV	4 метки противоречат данным своей строки (app_0005, app_0007, app_0009, app_0012); в материалах упомянуты только 0005 и 0007.
Порог БКИ	в 02 «< 500», в data_dictionary «< 550». Разметка CSV сделана по 550 (91/100 совпадений против 81 при 500) — берём 550.
05_api_schemas.yaml	образец `exclusiveMinimum: 0` — синтаксис OpenAPI 3.1, в файле 3.0.3 не проходит валидатор. Исправили на `minimum: 0, exclusiveMinimum: true`.
Эталон 07	для app_0001 «h = 2 → score = 720», по формуле из 06 — 640; DTI app_0003 указан 0,36, по формуле и в CSV — 0,18.

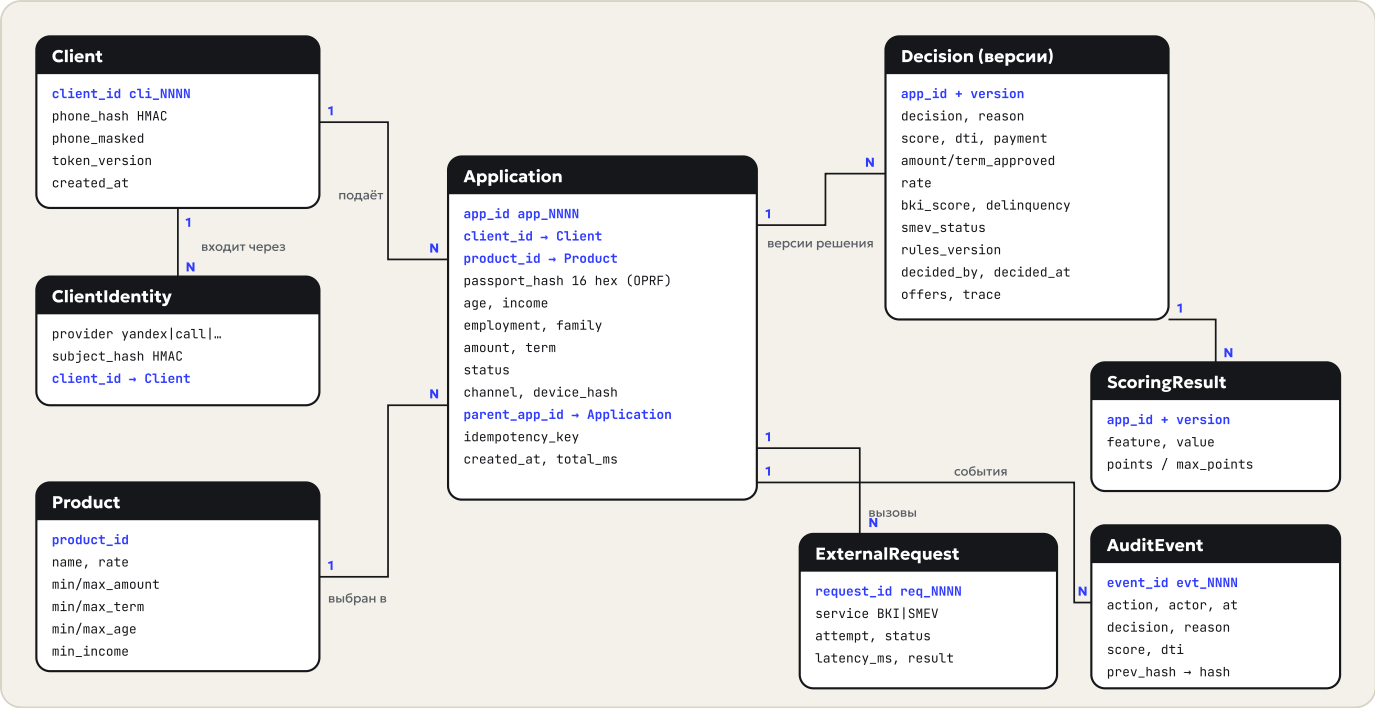
A.1. Схема системы



Блок	Что это	Что передаёт
Клиент	приложение банка (телефон или компьютер, Svelte)	JSON-заявку; паспорт — только как ослеплённую точку кривой для токенизатора, в заявку — отпечаток `passport_hash`
Кредитный конвейер	backend на Python (FastAPI): валидация, правила кейса (`credit_core`), оркестрация вызовов, решение	клиенту — решение, причину, разбивку балла, встречные предложения; в хранилище — версии решений; в аудит — события
Эмулятор БКИ	отдельный сервис на Go по спецификации кейса	`{score, delinquency, requests_90d}` по `md5(passport_hash) % 10`; отдельно — обезличенная история для ML-подсказки
Эмулятор СМЭВ	отдельный сервис на Go	`MATCH / MISMATCH` по `md5(client_id) % 10`; таймаут ставит конвейер
Токенизатор паспорта	изолированный Go-сервис, единственный владелец ключа OPRF (RFC 9497)	оценённую точку + доказательство; номера паспорта не видит никто
Хранилище	SQLite (в банке была бы PostgreSQL)	заявки, решения по версиям, разбивка балла, журнал вызовов; ПДн только хэшами и масками
Аудит	журнал с хэш-цепочкой, только добавление	event_id, app_id, action, decision, reason, score, dti, actor, at + хэши; подмена любой записи видна при проверке

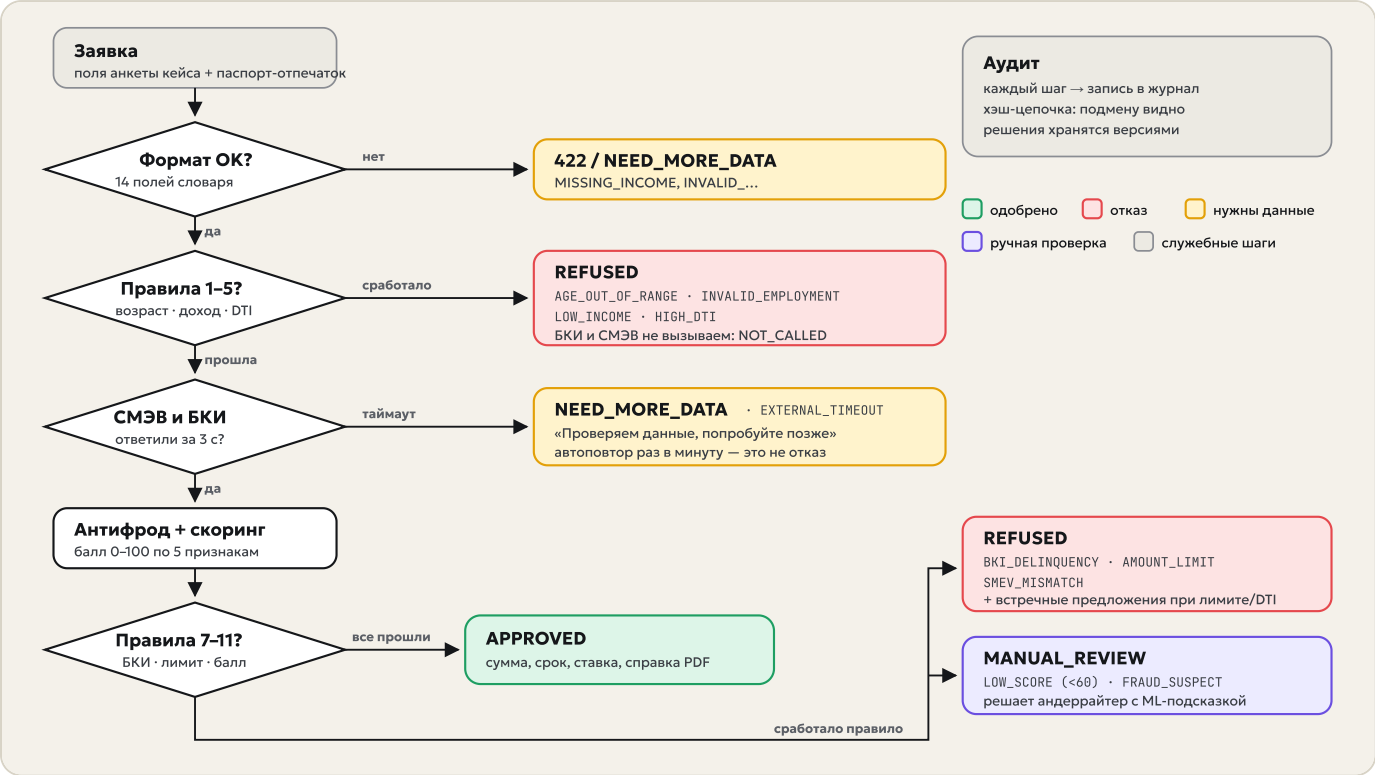
Потоки: заявка → валидация → правила 1–5 (без внешних вызовов) → СМЭВ и БКИ параллельно (HMAC-подпись запроса и ответа, 3 с × 2 попытки, каждая попытка в `logs/external.log`) → антифрод → скоринг → правила 6–11 → решение → аудит → уведомление клиенту по SSE.

A.2. ER-модель



Сущность	Назначение
Client	клиент банка; «личность» — проверенный телефон, хранится только HMAC-отпечаток и маска; ФИО и адреса нет
ClientIdentity	способы входа одного клиента (звонок; код поддерживает и OAuth-провайдеров): один телефон — один клиент
Product	кредитный продукт из products.csv: границы суммы, срока, возраста, дохода и ставка
Application	заявка: поля кейса, продукт, отпечаток паспорта, статус; встречное предложение — новая заявка со ссылкой на исходную
Decision	решение; при пересчёте или ручном решении добавляется новая версия, старые сохраняются; в каждом — версия правил
ScoringResult	разбивка балла по 5 признакам для каждой версии решения
ExternalRequest	каждая попытка вызова БКИ и СМЭВ: статус, задержка, краткий результат без ПДн
AuditEvent	журнал событий с хэш-цепочкой, append-only; проектная политика хранения — 5 лет

A.3. Схема принятия решения



Шаг	Что делаем	Если ошибка или сработало
1. Валидация	формат 14 полей, границы продукта	API: 422 с кодом и подсказкой; CSV: NEED_MORE_DATA
2. Правила 1–5	возраст, занятость, доход, мин. доход, DTI	REFUSED / NEED_MORE_DATA; БКИ и СМЭВ не вызываем (NOT_CALLED)
3. СМЭВ и БКИ	параллельно, HMAC, 3 с × 2 попытки	таймаут → NEED_MORE_DATA / EXTERNAL_TIMEOUT, автоповтор
4. Антифрод	граф связей по отпечаткам устройства, телефона, паспорта	только флаг для правила 11, отказа нет
5. Скоринг	балл 0–100, разбивка по признакам	итог < 0 → 0 и флаг score_capped_to_zero
6. Правила 6–11	таймаут, БКИ, лимит суммы, СМЭВ, балл, антифрод	REFUSED / NEED_MORE_DATA / MANUAL_REVIEW
7. Решение	параметры кредита, встречные предложения, справка	—
8. Аудит	запись каждого шага с хэшем предыдущей	сбой записи — заявка не теряется, решение пересчитывается

Статусы на выходе: `APPROVED` `REFUSED` `NEED_MORE_DATA` `MANUAL_REVIEW`. Принцип: при сомнении не одобряем, но и не отказываем из-за собственного сбоя — таймаут внешней системы даёт «нужны данные» и автоповтор.

B.1. Правила скоринга и отказа

Признак	Условие → баллы	Макс.	Обоснование
Доход	≥150k 30 · 80–150k 20 · 40–80k 10 · <40k 0	30	дано
Занятость	employed 25 · self-employed 20 · retired 10 · student 5 · unemployed 0	25	дано
Возраст	25–50 20 · 18–24 и 51–60 10 · 61–70 5	20	дано
Семья	married 15 · children 15 · single 10	15	два взрослых — запас прочности; за детей не штрафуем (нагрузку видно через DTI); разница 5 баллов
БКИ	≥700 10 · 600–699 5 · 500–599 0 · <500 или просрочка −20	10	история — лучший предсказатель; просрочка перекрывает почти любой плюс; итог <0 обрезаем до 0

Порог	Значение	Обоснование
Минимальный score	60	типичный надёжный клиент набирает 70–75; ниже 60 — несколько слабых мест. Ниже порога — ручная проверка (в разметке нет отказов по баллу)
Максимальный DTI	0,5 (2 знака)	на жизнь остаётся не меньше половины дохода; ЦБ считает нагрузку выше 50% высокой
Сумма ≤ X × доход	10	даже на 60 мес. под 18% платёж ≈ 25% дохода — «потолок» от слишком длинных крупных кредитов
Минимальный доход	30 000 ₽	чуть выше МРОТ 2026 (27 093 ₽); совпадает с min_income стандартного продукта
Возраст	18–70	дееспособность по ГК; после 70 доход и здоровье на весь срок не оценить
Отказ по БКИ	< 550 или просрочка	так в data_dictionary; только с ним правила совпадают с разметкой

Правила — сверху вниз, первое сработавшее даёт итог

N°	Условие	Решение	reason
1	age < 18 или > 70	REFUSED	AGE_OUT_OF_RANGE
2	employment = unemployed	REFUSED	INVALID_EMPLOYMENT
3	доход не указан	NEED_MORE_DATA	MISSING_INCOME
4	доход < 30 000 ₽	REFUSED	LOW_INCOME
5	DTI > 0,5	REFUSED	HIGH_DTI
6	БКИ/СМЭВ не ответили за 3 с	NEED_MORE_DATA	EXTERNAL_TIMEOUT
7	БКИ < 550 или просрочка	REFUSED	BKI_DELINQUENCY
8	сумма > 10 × доход	REFUSED	AMOUNT_LIMIT
9	СМЭВ = MISMATCH	REFUSED	SMEV_MISMATCH
10	балл < 60	MANUAL_REVIEW	LOW_SCORE
11	антифрод нашёл схему	MANUAL_REVIEW	FRAUD_SUSPECT
12	всё пройдено	APPROVED	—

DTI (02_правила, раздел 4): `r = 0,18 / 12; платёж = amount × r / (1 - (1 + r)^(-term)); dti = платёж / доход`; ставка 18% — для любого продукта. app_0001: 500 000 ₽ на 24 мес. → 24 962 ₽ → DTI 0,21, как в CSV (совпало в 99 из 99 строк).

B.2. Матрица «ситуация → reason → решение»

Ситуация	reason	Решение	Что видит клиент
age < 18 или > 70	`AGE_OUT_OF_RANGE`	REFUSED	Возраст не соответствует условиям продукта
employment = unemployed	`INVALID_EMPLOYMENT`	REFUSED	Кредит выдаётся при наличии работы, своего дела или пенсии
income пуст	`MISSING_INCOME`	NEED_MORE_DATA	Укажите доход
income < 30 000	`LOW_INCOME`	REFUSED	Недостаточный доход для данного продукта
DTI > 0,5	`HIGH_DTI`	REFUSED	Слишком высокая долговая нагрузка
BKI < 550 или delinquency	`BKI_DELINQUENCY`	REFUSED	Есть просрочки в кредитной истории
amount > 10 × income	`AMOUNT_LIMIT`	REFUSED	Сумма больше допустимой для вашего дохода
таймаут БКИ/СМЭВ	`EXTERNAL_TIMEOUT`	NEED_MORE_DATA	Проверяем данные, попробуйте позже
СМЭВ = MISMATCH	`SMEV_MISMATCH`	REFUSED	Данные не совпали при проверке, обратитесь в банк
балл < 60	`LOW_SCORE`	MANUAL_REVIEW	Заявку посмотрит специалист банка — обычно до одного рабочего дня
антифрод нашёл схему	`FRAUD_SUSPECT`	MANUAL_REVIEW	Заявку посмотрит специалист банка — обычно до одного рабочего дня
всё прошло успешно	—	APPROVED	сумма, срок, ставка, платёж, справка PDF

> При HIGH_DTI и AMOUNT_LIMIT клиент получает до 3 встречных предложений (меньше сумма, длиннее срок, другой продукт). Каждое прогоняется через те же правила с теми же ответами БКИ и СМЭВ, поэтому проходит проверенные правила. Предложения до получения БКИ помечаются preliminary и требуют полного конвейера. Про антифрод клиенту не сообщаем — текст тот же, что при ручной проверке.

Итог по 100 заявкам

Решение	Заявок
APPROVED	41
REFUSED	53
NEED_MORE_DATA	1
MANUAL_REVIEW	5

B.3. Тесты

Тест	Вход	Ожидание	Факт	Пройден
T2-1	app_0001 32 лет, 120000 ₽, employed, married, 500000 ₽ / 24 мес.	APPROVED, score ≥ 60, dti ≤ 0,5	APPROVED , балл 90, DTI 0,21	да
T2-2	app_0006 52 лет, 90000 ₽, retired, married, 400000 ₽ / 24 мес.	REFUSED, BKI_DELINQUENCY	REFUSED BKI_DELINQUENCY, балл 35, DTI 0,22	да
T2-3	app_0004 — лет, — ₽, employed, single, 200000 ₽ / 12 мес.	NEED_MORE_DATA, MISSING_INCOME	NEED_MORE_DATA MISSING_INCOME	да
T2-4	app_0002 19 лет, 15000 ₽, student, single, 300000 ₽ / 36 мес.	REFUSED, LOW_INCOME	REFUSED LOW_INCOME, балл 25, DTI 0,72	да
T2-5	app_0005 28 лет, 60000 ₽, self-employed, single, 800000 ₽ / 48 мес.	REFUSED, HIGH_DTI или AMOUNT_LIMIT	REFUSED AMOUNT_LIMIT, балл 65, DTI 0,39	да

T2-3: автоодобрения нет — балл и DTI не считаются, сумма не выдаётся. T2-5: DTI 0,39 ≤ 0,5, поэтому HIGH_DTI не срабатывает; срабатывает AMOUNT_LIMIT (800 000 > 600 000). В CSV стоит HIGH_DTI — это расхождение разобрано ниже. Клиенту в этом случае предлагается 600 000 ₽ на тот же срок.

Все расхождения с разметкой applications.csv

Заявка	В CSV	У нас	Объяснение (печатает сам прототип)
app_0005	REFUSED HIGH_DTI	REFUSED AMOUNT_LIMIT	В CSV HIGH_DTI, но DTI по формуле аннуитета 0,39 ≤ 0,5; срабатывает AMOUNT_LIMIT. Похоже на ошибку разметки (или DTI считали с другими кредитами, которых в данных нет).
app_0007	APPROVED	REFUSED SMEV_MISMATCH	В CSV одобрено, хотя СМЭВ вернул MISMATCH. Одобрять клиента, чей паспорт не подтвердился, нельзя — у нас отказ (правило 9).
app_0009	REFUSED AMOUNT_LIMIT	APPROVED	В CSV AMOUNT_LIMIT, но сумма 700000 ≤ 10 × доход 80000. Ни одно правило кейса не отказывает — ошибка разметки.
app_0012	REFUSED HIGH_DTI	APPROVED	В CSV HIGH_DTI, но DTI по формуле аннуитета 0,24 ≤ 0,5. Похоже на ошибку разметки (или DTI считали с другими кредитами, которых в данных нет).
app_0019	APPROVED	MANUAL_REVIEW LOW_SCORE	В CSV одобрено при балле 55 < 60. По условию кейса порог одобрения 60, поэтому автоматически не одобряем — заявку смотрит андеррайтер (правило 10).
app_0025	APPROVED	MANUAL_REVIEW LOW_SCORE	В CSV одобрено при балле 45 < 60. По условию кейса порог одобрения 60, поэтому автоматически не одобряем — заявку смотрит андеррайтер (правило 10).

app_0045	APPROVED	**MANUAL_REVIEW** LOW_SCORE	В CSV одобрено при балле 50 < 60. По условию кейса порог одобрения 60, поэтому автоматически не одобряем — заявку смотрит андеррайтер (правило 10).
app_0064	APPROVED	**MANUAL_REVIEW** LOW_SCORE	В CSV одобрено при балле 45 < 60. По условию кейса порог одобрения 60, поэтому автоматически не одобряем — заявку смотрит андеррайтер (правило 10).
app_0070	APPROVED	**MANUAL_REVIEW** LOW_SCORE	В CSV одобрено при балле 55 < 60. По условию кейса порог одобрения 60, поэтому автоматически не одобряем — заявку смотрит андеррайтер (правило 10).

Как ещё проверяли

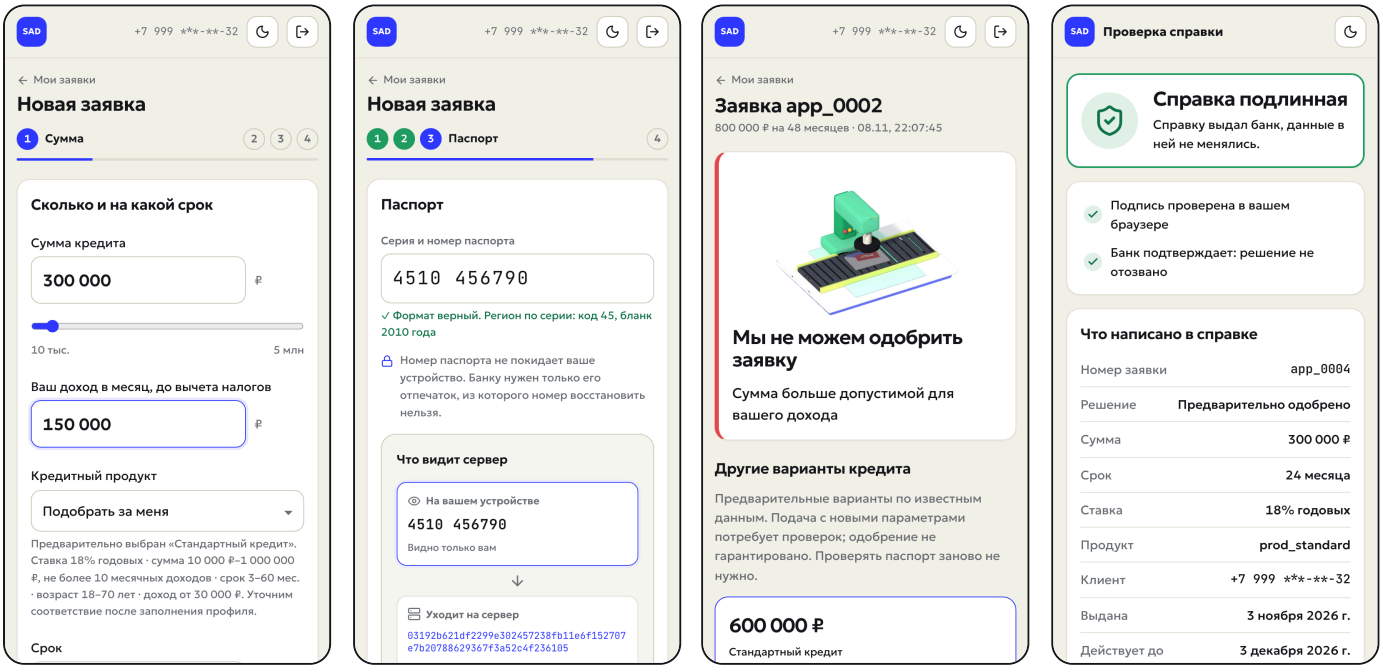
260 автотестов	правила (все ветки, порог 550, обрезка балла до 0), API, роли по матрице, видимость полей, справка и её отзыв, OPRF-прокси, таймауты и автоповтор, защита входа, подписи, цепочка аудита; тест паритета Go ↔ Python на 1000 значениях
18 из 18 живых сценариев	скрипт против запущенной системы; включая повторную заявку: проверка дублей включена, все сценарии выполнены на отдельной базе
Ручная проверка	путь клиента на телефоне и экраны сотрудников в браузере; безопасность вручную — Александр Чиранов

B.4. Интеграция, безопасность и экраны

Внешний сервис	Контракт	Таймаут и сбои
БКИ	`POST /bki/score {passport_hash} → {score, delinquency, requests_90d}`	3 с, 2 попытки (повтор на таймаут и 5xx), затем NEED_MORE_DATA / EXTERNAL_TIMEOUT и автоповтор раз в минуту; неверная подпись ответа = ошибка + событие безопасности
СМЭВ	`POST /smev/check {client_id, passport_hash} → {status}`	

Угроза	Мера	Остаток
U-1 дискриминация	пола, национальности, ФИО, адреса нет в данных; разница по семье — 5 баллов	**низкий**
U-2 утечка ПДн	паспорт не покидает устройство (OPRF); телефон — HMAC; в логи и аудит не пишутся паспорт и телефон; согласие на обработку данных — обязательный чекбокс, факт фиксируется в аудите	**низкий**
U-3 подделка заявки	СМЭВ + антифрод (граф связей, kNN к схемам + IsolationForest) → ручная проверка	**средний**
U-4 подбор суммы	расчёт до подачи без БКИ; одна активная заявка на паспорт; лимит частоты	**низкий**
U-5 подмена аудита	хэш-цепочка, решения по версиям, удаления нет ни у одной роли	**низкий**
U-6 таймаут/ подмена ответа	таймауты, HMAC запроса и ответа с меткой времени	**низкий**
U-7 захват аккаунта	клиент без паролей; сотрудники — bcrypt, блокировка после 5 ошибок, JWT 60 мин	**низкий**

RBAC: клиент — подать заявку и видеть только свои; сервис — пересчёт после таймаута; андеррайтер — ручное решение и пересчёт; аудитор — только чтение журнала; менять правила не может никто через интерфейс (только код и ревью, версия правил в каждом решении); удалять аудит не может никто.



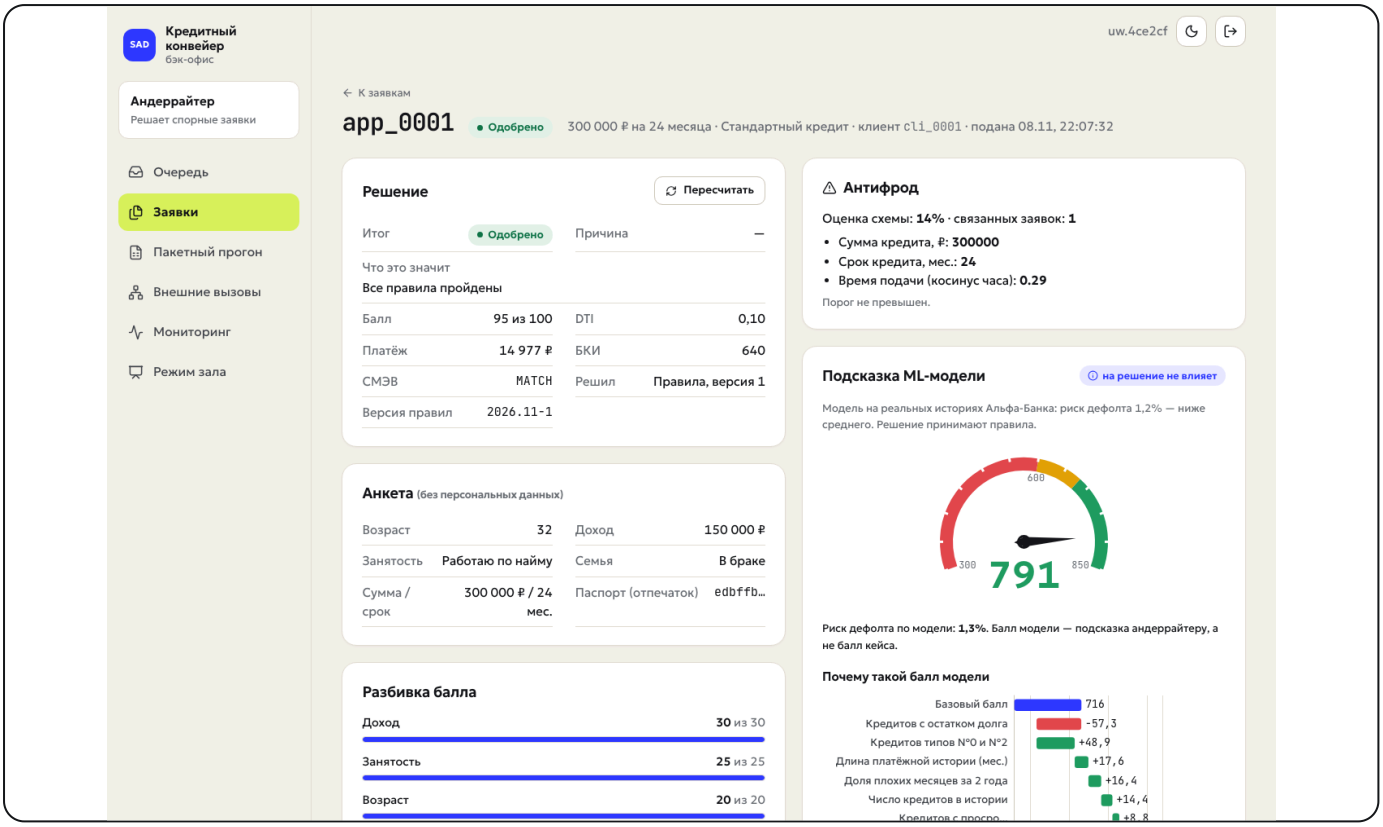
Проверка подписи справки

Расчёт до подачи и автоподбор продукта

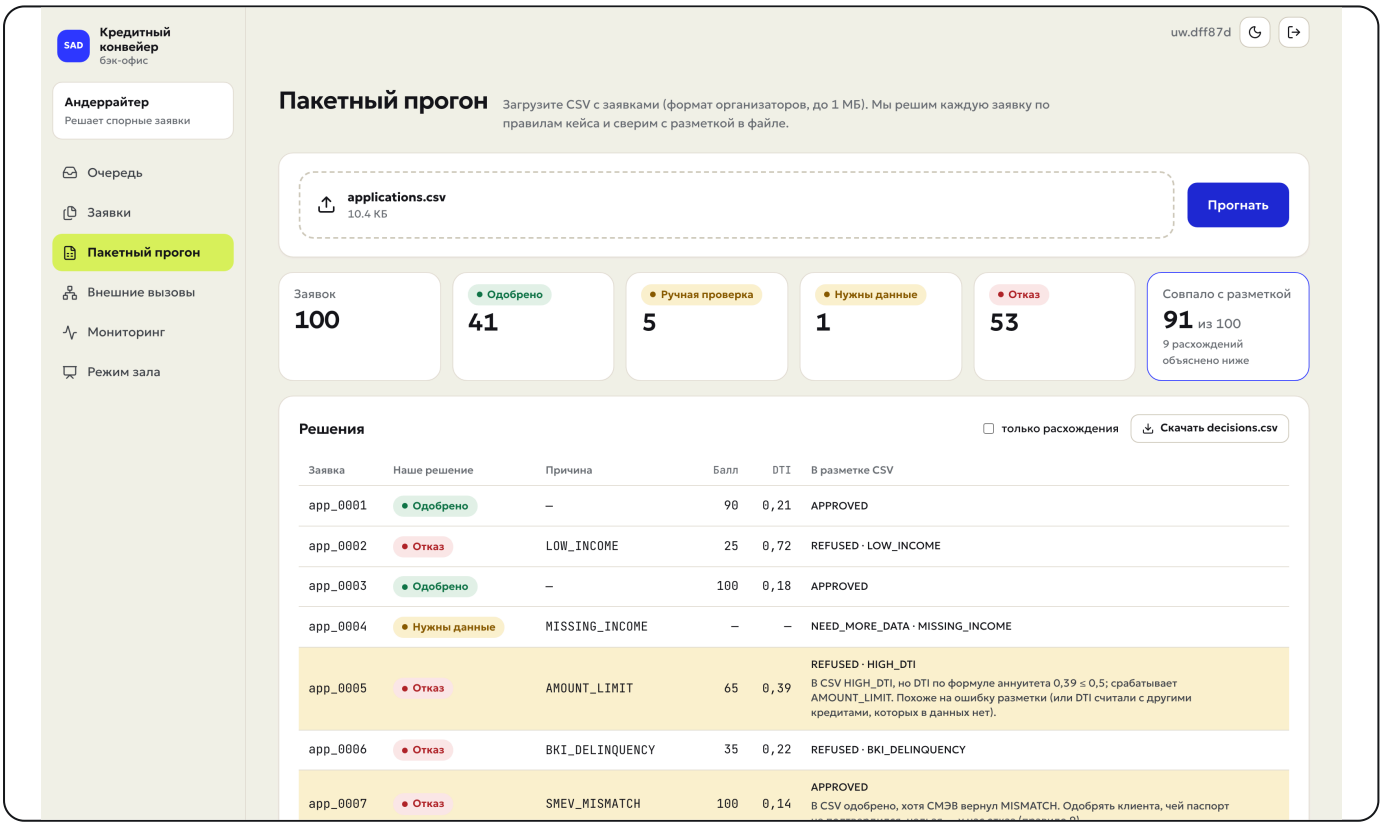
«Что видит сервер»: только случайные байты

Отказ AMOUNT_LIMIT и встречное предложение

Экраны бэк-офиса



Карточка андеррайтера: решение и причина, ручное решение, антифрод, ML-подсказка на реальной истории Альфа-Банка (на решение не влияет)



Пакетный прогон applications.csv прямо в интерфейсе: 91 из 100 совпадений с разметкой, каждое расхождение объяснено, скачивание decisions.csv

С. Журнал промптов

ИИ-ассистенты (Claude Code и Codex) помогали писать код по нашим требованиям и черновики текстов; каждый результат проверяли запуском, тестами и сверкой цифр. Решения — кейс, стек, правила, архитектура, безопасность, способы входа, дизайн — принимала команда. 3D-сцены для интерфейса собраны в Blender вручную, без ИИ.

Дата	Цель	Промпт (сжато)	Результат	Проверка	Использовано
31.10	Выбор кейса	«сравни 4 кейса чемпионата по объёму работ и рискам, помоги выбрать»	Таблица плюсов и минусов по каждому кейсу	Сверили с требованиями организаторов, решили сами	да
31.10	Каркас проекта	«собери каркас репозитория: FastAPI + SQLite, Svelte без React, Go для эмуляторов БКИ/СМЭВ, приложи черновик контрактов API»	Структура репозитория, черновик контрактов	Контракты прочитали, роли и правила переписали своими словами	переработано
31.10	Модель данных	«поля заявки есть в data_dictionary организаторов, спроектируй таблицы SQLite: версии решений, разбивка балла, аудит с хэш-цепочкой»	Схема таблиц, миграции	Сверили каждую колонку со словарём данных	да
31.10	Валидация	«реализуй проверки полей: возраст 18-70, доход > 0, справочники занятости и семьи, границы продукта»	credit_core/validation.py	Прогнали на 100 строках applications.csv: ошибки формата совпали с ручной проверкой	да
01.11	Правила скоринга	«веса дохода, занятости и возраста даны условием, family берём 15/15/10, БКИ 10/5/0/-20, запиши в rules.py и накрой тестами»	credit_core/rules.py + тесты весов	Посчитали app_0001 вручную (балл 90), сверили с кодом	да
01.11	Порядок правил	«правила анкеты проверяем до внешних вызовов, таймаут это не отказ, балл меньше 60 это ручная проверка. Реализуй engine.py с трассировкой каждого правила»	credit_core/engine.py, трассировка	Перебрали пороги на 100 заявках: 91/100 совпадений с разметкой при пороге БКИ 550	да
01.11	Консольный прототип	«напиши 04_прототип.py: читает applications.csv, пишет decisions.csv, печатает сводку и объяснение каждого расхождения»	prototype/04_прототип.py с режимом --explain	Прогон 100 заявок; app_0001, 0002, 0005, 0006 пересчитали руками — сходится	да
01.11	Вход клиента	«сравни вход по номеру телефона без паролей: Telegram-бот, звонок SMS.RU, Яндекс ID с PKCE. Цена и риски?»	Сравнение способов входа, реализация четырёх	Проверили на своих номерах; платные SMS отклонили	переработано
01.11	Антифрод	«граф связей заявок по отпечаткам устройства/телефона/паспорта + IsolationForest, только флаг для ручной проверки, без автоблокировок»	backend/app/antifraud/	Контрольный прогон на симуляции: 80% схем поймано, ложных тревог 0–3%	да

02.11	Сверка с кейсом	«нашли папку с шаблонами организаторов, проверь наш сервис по их чек-листу, что обязательно переделать»	Список: балл 0–100, CLI, эмуляторы по спецификации кейса, роли	Шаблоны читали сами, план переделки утвердили сами	да
02.11	Эмуляторы на Go	«вынеси БКИ и СМЭВ в отдельные Go-сервисы с HMAC-подписью запроса и ответа, сделай тест совпадения формул с Python»	services/cmd/bki, services/cmd/smev	Тест паритета на 1000 значениях — формулы совпали; вручную подделывали подпись	да
02.11	OPRF-токенизатор	«паспорт не должен покидать браузер, реализуй OPRF (RFC 9497) отдельным Go-сервисом, ключ только там»	services/cmd/tokenizer	Проверили логи: на сервере только ослеплённые байты, номера нет	да
02.11	Аудит безопасности	«проверь код глазами взломщика: вход, роли, подписи внешних систем, журнал, справка»	22 находки с воспроизведением	Каждую находку воспроизводили до исправления, потом тест	да
02.11	RBAC и журнал	«вот наша матрица прав: клиент видит только своё, аудитор только читает, удалений аудита нет. Проверь каждую ручку и добавь 403 в аудит»	Проверка ролей на всех ручках, тест матрицы	Прогнали под каждой ролью вручную и тестом	да
02.11	Тёмная тема и мобильная сетка	«палитра и макеты готовы, собери тёмную тему на CSS-переменных и адаптивную сетку для экранов клиента»	frontend/src/app.css, сетка экранов	Просмотрели все экраны на телефоне и десктопе	переработано
03.11	Разбор расхождений CSV	«прототип даёт 91/100, вот 9 строк с расхождениями, проверь наши объяснения по каждой»	Подтверждение расчётов: 4 ошибки разметки, 5 заявок с баллом < 60	Каждую строку пересчитали вручную (DTI по формуле, лимит 10×)	да
03.11	Документы кейса	«заполни шаблоны организаторов из нашего кода: 02_правила, 03_тесты, data_dictionary, 06_external_services»	Черновики заполненных шаблонов	Каждую цифру сверили с выводом прототипа и тестами; OpenAPI прогнали через валидатор	переработано
03.11	Презентация и приложение	«собери 8 слайдов по структуре из инструкции чемпионата, наш текст и цифры не менять»	HTML-слайды и приложение A–E	Вычитали сами, часть формулировок заменили	переработано
06.11	Состав команды	«в документах указаны не все участники: добавь Игнатьева и Иванова в титул и матрицу вклада (5 человек), речь на двоих, ветка team_10_case_2»	Правки титула, речи, матрицы вклада, раздела E	Перечитали каждый изменённый файл, пересобрали PDF	да
06.11	Финальная сверка	«сверь цифры во всех документах с тестами и прототипом: автотесты, живые сценарии, совпадения с разметкой, находки аудита»	Сверка чисел во всех документах	258 автотестов зелёные, прототип 91/100, сайт отвечает	да
08.11	Обновление материалов	«организаторы 07.11 обновили файлы на Диске: сверь наш комплект с новыми application.csv, product.csv и 02_правила; переведи систему на каталог из 3 продуктов и DTI по ставке 18%»	Список расхождений, правки ядра, тестов и документов	Сравнили CSV построчно (100 из 100 совпали), прототип снова 91/100, decisions.csv побайтно тот же; все автотесты зелёные	переработано

D. Матрица вклада

Участник	Файлы / задачи	Часы	Подпись
Рахматуллин Динар	архитектура и контракты, ядро правил credit_core, консольный прототип, backend и API, ML-модель (CatBoost на данных Альфа-Банка), антифрод, Go-сервисы, вход клиента, деплой на credit.fashf.ru; 02_правила, 04_requirements, 05_api_schemas, 06_external_services, 07_decision_mechanism, 08_reasons, data_dictionary; слайды 1–5 и демо	20	
Чиранов Александр	информационная безопасность: модель угроз и меры (ИБ.md), RBAC и аудит; ручная проверка всех защитных сценариев (подбор пароля, роли, подмена подписи, подделка аудита и справки, OPRF); слайды 7–8	12	
Коломеец Софья	3D-сцены (живой конвейер, штамп решения) в Blender — несколько итераций; дизайн интерфейса и слайдов; схемы системы и решения (A.1, A.3)	12	
Игнатьев Дмитрий	тестирование: факты в 03_тесты по прогону прототипа и сервиса; автотесты (260) и живые сценарии (18/18 на отдельной базе); ручное тестирование интерфейса на телефоне и десктопе — около 20 найденных и исправленных багов; сверка решений прототипа с разметкой (91/100)	10	
Иванов Юрий	интерфейс клиента на Svelte: расчёт и подбор кредита, анкета, экран решения и встречных предложений, личный кабинет; интеграция с API и SSE-обновлениями; Docker/nginx-сборка и локальный стенд scripts/run.sh	10	

E. Артефакты

Ревизия исходников 82c6772. Ветка для сдачи: `team_10_case_2` (команда №10 в кейсе 2).

```
$ git log -1 --format="commit %H%nAuthor: %an%nDate:   %ad%n%n    %s"
commit 82c6772406bee9f46290e8143f89706422c5afbb
Author: fashfffffff
Date:   Sun Nov 8 20:29:20 2026 +0300

    ИБ: вход клиента на стенде — звонок и демо-код, OAuth и Telegram выключены
```

E.1. Последний коммит; ветки main и team_10_case_2 на GitHub указывают на него

```
$ python3 prototype/04_прототип.py data/case2/applications.csv --out decisions.csv
OK: decisions.csv   (правила 2026.11-1, БКИ и СМЭВ: из CSV)
    обработано: 100
    решения:
      ✓ APPROVED          41
      ✗ REFUSED           53
      ? NEED_MORE_DATA     1
      ◐ MANUAL_REVIEW      5
    причины:
      INVALID_EMPLOYMENT     18   Отказ
      HIGH_DTI               12   Отказ
```

E.2. Прогон 100 заявок. Результат побайтно совпадает с decisions.csv в комплекте.

```
$ md5 -r data/case2/applications.csv
347f3b74694eb41a6735a4e4a12aa505 data/case2/applications.csv
```

E.3. MD5 файла applications.csv. Хэш совпадает с файлом 01_данные/application.csv из папки организаторов (версия от 07.11): данные не менялись.